

System Reference Guide

MalScan AI

Detecting malicious software by examining a file's structure, without ever running it

Tamal Mishra

BCA (Cybersecurity) · EThames Business School, Hyderabad
EDC ID EDC-2024-189 · OU ID 1289-24-861-024

73 features · 27 detection rules · 12,000 training rows

1 What MalScan AI is

MalScan AI looks at a file and decides whether it is malicious — without ever opening or running it.

It works the way a forensic examiner would: by studying the structure of the thing rather than its behaviour. It measures 73 separate properties of a file — how its internal sections are arranged, which Windows functions it borrows, how random its contents look, what text is buried inside it — and feeds those measurements to a model trained to recognise the patterns malicious files tend to share.

The result is a probability, a verdict of **Benign**, **Suspicious** or **Malicious**, and — importantly — a breakdown of exactly which properties pushed the score in which direction.

73

properties measured per file

27

pattern rules applied

0.9963

ROC-AUC on held-out test data

< 1 s

typical scan of a Windows program

The problem it addresses

Traditional antivirus works by signature: a catalogue of fingerprints of known malware. It is fast and precise, but it can only recognise what someone has already found and added to the list. That leaves three predictable gaps:

- **Zero-day malware** — brand new, so nobody has catalogued it yet.
- **Polymorphic malware** — rewrites itself between infections, so its fingerprint changes every time.
- **Packed malware** — compressed or encrypted so its bytes look different in every build.

A model that judges *structure* does not need to have seen the specific file. It recognises the traits malicious programs tend to share: randomness suggesting hidden content, an import table that has obviously been stripped, memory regions that are writable and executable at the same time, function combinations that only make sense if you intend to inject code into another process.

An honest boundary. This complements signature scanning; it does not replace it. A signature scanner is near-perfect on malware it already knows. MalScan is designed for the cases it does not.

2 How a scan works, step by step

Every scan runs the same nine stages. The whole sequence takes under a second for a typical Windows program.

#	Stage	What happens	Code
1	File type detection	Reads the first few bytes of the file and matches them against known signatures. The extension is deliberately ignored, because anyone can rename <code>virus.exe</code> to <code>holiday.jpg</code> ; the bytes at the start of a file are far harder to fake.	<code>features/filetype.py</code>
2	Hashing	Computes MD5, SHA-1 and SHA-256 fingerprints, plus a similarity digest that stays close for near-identical files, so repackaged variants of the same sample can be spotted.	<code>features/hashes.py</code>
3	Structural analysis	If the file is a Windows program, its headers, section table, imports, exports and resources are parsed out and examined for anomalies.	<code>features/pe_features.py</code>
4	Randomness measurement	Measures entropy for the whole file, for each section and for every 4 KB block. Ordinary code and text are predictable; compressed or encrypted content is not, so a high reading suggests something is hidden.	<code>features/entropy.py</code>
5	Text mining	Pulls readable text out of the binary and counts web addresses, IP addresses, registry paths, file paths, suspicious keywords, encoded blocks and any executables hidden inside.	<code>features/strings_features.py</code>
6	Rule matching	Runs 27 YARA rules over the file. Each carries a category and a severity from 1 to 5. The number of hits and the worst severity become two of the 73 features.	<code>rules/engine.py</code>
7	Classification	The 73 numbers are handed to the trained model, which returns a probability that the file is malicious. Files without a usable Windows structure are routed to a content scorer instead.	<code>model/predict.py</code>
8	Explanation	Each feature in turn is reset to its typical benign value and the model is asked again. How far the score moves is that feature's contribution to this particular verdict.	<code>model/explain.py</code>
9	Recording	The verdict, the evidence and the hashes are written to SQLite. The file itself is never stored.	<code>storage/history.py</code>

Nothing is ever executed. At no point is the file written somewhere it could run, handed to a command shell, loaded as a library or launched. It is read into memory as a plain block of bytes and taken apart as data. That is what makes it safe to examine a genuine malware sample.

3 What each technology does and why it was chosen

Every component was chosen for a specific reason. This table is the direct answer to "what did you use, and why that?"

Technology	What it does	Why this one
Python 3.9+ running 3.13.15	The language the whole system is written in.	It is the only ecosystem with both mature binary-parsing libraries and mature machine-learning libraries. Using one language for the parser, the model and the interface removes every translation layer between them.
Streamlit	Turns the Python code into a web interface: upload box, tabs, charts, tables.	It renders a browser UI directly from Python, so there is no separate front-end project, no JavaScript build step and no API layer to keep in sync. For a single developer that removes an entire tier of work.
pefile	Reads the internal structure of a Windows program: headers, sections, imports, exports, resources.	It is pure Python and treats the file strictly as a data structure. It never asks Windows to load the program, which is exactly the property a malware scanner needs.
YARA yara-python	Matches the file against 27 written rules describing known malicious patterns.	YARA is the industry standard for malware pattern rules and is used by VirusTotal and most commercial vendors. Its rule syntax is readable, so the detection logic can be inspected and extended by hand.
scikit-learn	Provides the Random Forest classifier, the probability calibration and every evaluation metric.	It is the reference toolkit for classical machine learning, with the grouped splitting and calibration methods this project depends on built in and well tested.
XGBoost	A second, competing classifier (gradient boosting).	Training two different algorithms and selecting between them on measured performance is more defensible than asserting one is better. Both are trained every run and the comparison is recorded.
NumPy	Holds the feature vectors as numeric arrays and powers the explanation step.	The explainer builds 74 variants of the file's feature vector and scores them in one batch. NumPy makes that a single fast operation instead of a loop.
pandas	Organises the training set and every table shown in the interface.	It is the standard way to handle labelled tabular data in Python and reads and writes the CSV/Parquet training files directly.
Plotly	Draws the charts: contribution bars, entropy graphs, ROC curves, dashboards.	Its charts are interactive inside Streamlit, so values can be hovered and inspected rather than just looked at.
SQLite	Stores the history of every scan.	It ships inside Python, needs no server, no installation and no configuration, and keeps the whole database in one file that can be copied or deleted.
joblib	Saves the trained model to disk and loads it back.	It is the format scikit-learn itself recommends, and it handles the large numeric arrays inside a trained forest efficiently.

Technology	What it does	Why this one
python-magic optional	Identifies file types from their content.	Optional by design. If it is missing, MalScan falls back to its own magic-byte table, so a missing system library degrades the result instead of stopping the program.

Designed to degrade, not to fail

Two of these are optional on purpose. Without `yara-python` a built-in pure-Python matcher handles the plain-text patterns from the same rule file. Without `python-magic` the bundled magic-byte table identifies file types. In both cases the program keeps working with reduced capability instead of refusing to start, and the sidebar always states which backend is actually live — so the user is never misled about what is running.

4 The 73 properties measured

These are the "extracts" — every measurement taken from a file, in the exact order the model receives them. They are grouped into six families. The descriptions below are the same words shown to the user in the interface.

File 11 features

#	Internal name	What it measures
1	file_size_log	How big the file is
2	entropy_overall	How random the whole file looks, from 0 to 8 - high means packed or encrypted
3	entropy_first_block	How random the opening 4 KB looks, where the file's header sits
4	entropy_last_block	How random the final 4 KB looks, where extra data tends to get tacked on
5	printable_ratio	How much of the file is readable text
6	null_ratio	How much of the file is empty padding
7	byte_chi2	How evenly the byte values are spread - a very even spread suggests encryption
8	is_pe	The file is a Windows program
9	is_script	The file is a script or plain text
10	is_document	The file is a document, such as Office or PDF
11	is_archive	The file is a compressed archive, such as a ZIP

PE Header 22 features

#	Internal name	What it measures
1	pe_is_64bit	Built for 64-bit Windows
2	pe_is_dll	It is a shared library rather than a program you run directly
3	pe_num_sections	How many parts the program is divided into
4	pe_timestamp_plausible	The build date on the file looks believable
5	pe_timestamp_age_years	How many years ago the file claims it was built
6	pe_size_of_code_log	How much of the file is actual program code
7	pe_size_of_image_log	How much memory the program asks for when it loads
8	pe_size_of_headers	How much room the file's own headers take up
9	pe_checksum_valid	The file's built-in integrity check still matches, so it has not been edited since it was built
10	pe_subsystem_gui	Runs as a windowed application

#	Internal name	What it measures
11	pe_subsystem_console	Runs in a console window
12	pe_aslr	Keeps the standard defence that loads the program at a random address
13	pe_dep	Keeps the standard defence that stops data being run as code
14	pe_seh	Leaves Windows' normal error handling switched on
15	pe_has_debug	Still carries leftover debugging information
16	pe_has_relocs	Can be loaded at a different address than it prefers
17	pe_has_tls	Runs hidden setup code before the program's real starting point, which is sometimes used to dodge analysis
18	pe_has_resources	Bundles resources such as icons or images
19	pe_has_signature	Carries a digital signature from whoever published it
20	pe_has_version_info	States a publisher, product name and version
21	pe_ep_in_code_section	Starts running inside the normal code area, as a legitimate program does
22	pe_ep_section_entropy	How random the code looks at the exact point the program starts

Sections 10 features

#	Internal name	What it measures
1	sec_entropy_mean	Average randomness across every part of the program
2	sec_entropy_max	Randomness of the most scrambled-looking part
3	sec_entropy_min	Randomness of the least scrambled-looking part
4	sec_entropy_std	How much the randomness varies from part to part
5	sec_high_entropy_count	How many parts look packed or encrypted
6	sec_wx_count	Parts the program can both rewrite and run, which is how self-modifying code behaves
7	sec_zero_raw_count	Parts that reserve memory but hold nothing on disk, so they get filled in while running
8	sec_nonstandard_names	Parts with unusual names that normal build tools do not produce
9	sec_vsize_ratio_mean	Average gap between the memory a part claims and what it actually holds
10	sec_vsize_ratio_max	The largest such gap - a wide one suggests the file unpacks itself

Imports 13 features

#	Internal name	What it measures
1	<code>imp_dll_count</code>	How many Windows libraries the program borrows from
2	<code>imp_func_count</code>	How many Windows functions the program borrows
3	<code>imp_has_kernel32</code>	Uses core Windows functions for files, memory and processes
4	<code>imp_has_advapi32</code>	Uses the Windows registry or system services
5	<code>imp_has_network</code>	Is able to communicate over the network
6	<code>imp_has_user32</code>	Can create windows, or watch keyboard and mouse input
7	<code>imp_has_crypt</code>	Uses Windows encryption functions
8	<code>imp_has_shell</code>	Can launch other programs or open files
9	<code>imp_suspicious_count</code>	How many of the functions it borrows are on our watchlist
10	<code>imp_suspicious_ratio</code>	What share of the functions it borrows are on our watchlist
11	<code>imp_dynamic_resolution</code>	Looks its functions up while running instead of declaring them upfront, which is a common way to hide what a program intends to do
12	<code>imp_table_sparse</code>	Declares far fewer functions than a file this size normally would, which is typical of packed files
13	<code>exp_count</code>	How many functions this file offers to other programs

Content 12 features

#	Internal name	What it measures
1	<code>str_count_log</code>	How many readable pieces of text the file contains
2	<code>str_avg_len</code>	Average length of those pieces of text
3	<code>str_max_len</code>	Length of the longest piece of text
4	<code>str_max_entropy</code>	How scrambled the most random piece of text looks
5	<code>str_urls</code>	How many web addresses are buried in the file
6	<code>str_ips</code>	How many IP addresses are buried in the file
7	<code>str_registry</code>	How many Windows registry locations are named
8	<code>str_paths</code>	How many file or folder locations are named
9	<code>str_suspicious_kw</code>	How many words from our watchlist turn up
10	<code>str_base64_blobs</code>	Long encoded blocks that may be a hidden payload
11	<code>str_has_pdb</code>	Still carries the build path from the original developer's machine

#	Internal name	What it measures
12	<code>str_embedded_mz</code>	Whole other programs hidden inside this one

Packing & Rules 5 features

#	Internal name	What it measures
1	<code>overlay_ratio</code>	How much data is tacked on past the program's proper end
2	<code>packer_hit</code>	Matches a known packer such as UPX, meaning the real contents have been compressed out of sight
3	<code>rule_hits</code>	How many detection rules this file set off
4	<code>rule_max_severity</code>	How serious the worst rule it set off was, from 0 to 5
5	<code>res_entropy_max</code>	How scrambled the most random bundled resource looks

Why a fixed order matters. The list lives in one file, `features/schema.py`, and the extractor, the model and the explainer all read it from there. A model trained against a different list is rejected on load rather than being allowed to produce nonsense from mismatched columns.

5 The 27 detection rules

Alongside the model, the file is matched against 27 hand-written YARA rules describing known malicious techniques. Each has a category and a severity from 1 to 5.

A rule hit is evidence, not a verdict. Ordinary Windows software legitimately uses many of these same functions. That is exactly why rule hits enter the model as two features among 73 rather than deciding the answer by themselves.

anti-analysis 3 rules

Sev	Rule	What it looks for
4	AntiVM_Environment_Checks	Looks for virtual machine or sandbox artefacts
4	AntiAnalysis_Tool_Detection	Checks for the presence of analysis tools
3	AntiDebug_API_Checks	Anti-debugging API checks

c2 1 rule

Sev	Rule	What it looks for
3	C2_Hardcoded_IP_Endpoint	Hardcoded IP address plus socket APIs

credential-theft 1 rule

Sev	Rule	What it looks for
5	Theft_Credential_Access	Targets stored credentials or LSASS memory

downloader 1 rule

Sev	Rule	What it looks for
4	C2_Network_Download_Execute	Downloads a file and executes it

injection 3 rules

Sev	Rule	What it looks for
5	Injection_Classic_RemoteThread	Classic remote-thread process injection API sequence
5	Injection_Process_Hollowing	Process hollowing / RunPE indicators
4	Injection_APC_Queue	APC-injection primitives

lolbin 2 rules

Sev	Rule	What it looks for
4	LOLBin_PowerShell_Encoded	Encoded or hidden PowerShell command line
4	LOLBin_Script_Download_Cradle	Script download-and-run cradle

maldoc 2 rules

Sev	Rule	What it looks for
4	Document_Macro_AutoExec	Office document with auto-executing macro entry points
3	PDF_Suspicious_Action	PDF containing JavaScript or an auto-launch action

obfuscation 3 rules

Sev	Rule	What it looks for
3	Injection_Dynamic_API_Resolution	Resolves APIs at runtime while importing almost nothing
3	Anomaly_Reversed_Strings	API names stored reversed to defeat string scanning
2	Obfuscation_Base64_Payload	Long base64 blob embedded in the file (possible packed payload)

packing 2 rules

Sev	Rule	What it looks for
3	Packer_Commercial_Protector	Commercial protector or virtualiser (Themida/VMProtect/ASPack/Enigma)
2	Packer_UPX	UPX-packed executable

persistence 2 rules

Sev	Rule	What it looks for
4	Persistence_Registry_Run_Key	Writes to a Run key for persistence
3	Persistence_Scheduled_Task_Or_Service	Creates a scheduled task or a Windows service

privilege 1 rule

Sev	Rule	What it looks for
3	Privilege_Token_Manipulation	Adjusts process privileges (e.g. SeDebugPrivilege)

ransomware 3 rules

Sev	Rule	What it looks for
5	Ransomware_Shadow_Copy_Deletion	Deletes volume shadow copies to block recovery

Sev	Rule	What it looks for
5	Ransomware_Note_Language	Extortion / ransom note wording
4	Ransomware_Crypto_Primitives	Bulk file enumeration combined with encryption APIs

spyware 1 rule

Sev	Rule	What it looks for
4	Theft_Keylogging	Keyboard or clipboard capture primitives

structure 1 rule

Sev	Rule	What it looks for
3	Anomaly_Embedded_Executable	A second PE executable is embedded inside the file

test 1 rule

Sev	Rule	What it looks for
5	Test_EICAR_Signature	EICAR anti-malware test file (harmless, used to verify scanners)

6 The machine learning, explained plainly

What the model actually is

A **Random Forest** is a large collection of decision trees. Each tree asks a sequence of yes/no questions about the numbers it is given — "is the entropy above 7.4?", "are there fewer than eight imports?" — and arrives at a guess. This model contains 400 such trees, and the forest's answer is the vote across all of them. Tree-based models were chosen because the features are full of exactly those natural cut-off points, which trees handle directly and without any rescaling of the data.

Where the training data came from

This is the part that most deserves scrutiny, and it is stated openly rather than buried.

The benign half is measured, not invented. 1,000 real Windows programs from this machine's system folders were passed through the *same* extractor the scanner uses live. So the awkward realities of genuine software survive into the data instead of being idealised away.

The malicious half is synthesised from documented behaviour profiles. A live malware corpus cannot be hosted for a student project. Instead each synthetic sample *starts as a real benign file's measurements* and then has a family profile applied, shifting only those properties that malware research consistently finds meaningful. Six families are modelled: `packed_dropper` , `ransomware` , `rat_backdoor` , `spyware_keylogger` , `injector_loader` , `trojanized_installer` .

That shared real backbone is what makes the exercise meaningful. Generating malicious rows from nothing would let the model separate the classes on artefacts of the generator rather than on anything security-relevant.

Dataset property	Value
Total rows	12,000
Benign / malicious	6,000 / 6,000
Real source binaries	1,000
Deliberately hard benign rows	720 (packed but legitimate)
Deliberately subtle malicious rows	877 (partial profile only)
Train / test rows	9,529 / 2,471
Split method	GroupShuffleSplit by source binary
Source files spanning the split	0 (asserted by the trainer)

Why the split is grouped. Several rows descend from the same original binary. Splitting at random would scatter near-duplicates across both sides, so the model would be tested on files it had effectively already seen and every score would be inflated. Splitting by source binary prevents that, and the trainer refuses to proceed if any file appears on both sides.

Choosing between the two models

Both algorithms are trained every run and compared on held-out data.

Measure	Random Forest	XGBoost	What it means
Accuracy	0.9842	0.9887	Share of test files judged correctly
Precision	0.9884	0.9958	When it says malicious, how often it is right
Recall	0.9795	0.9811	Share of malicious files it actually caught
F1	0.9839	0.9884	Balance of precision and recall
ROC-AUC	0.9963	0.9962	Ranking quality at any threshold — the selection criterion
False alarm rate	0.0112	0.004	Clean files wrongly flagged
Miss rate	0.0205	0.0189	Malicious files wrongly cleared

RandomForest was selected on ROC-AUC, which measures how well the model ranks files regardless of where the cut-off is placed — the right criterion here, because the threshold is adjustable at scan time. Note that XGBoost was actually better on raw accuracy; the comparison is recorded rather than quietly dropped.

Cross-validation over 5 grouped folds gave a mean ROC-AUC of **0.9961** (standard deviation 0.0006), so the result is stable rather than a lucky split.

Calibration: making the percentage mean something

If 70% of the trees vote "malicious", that is *not* the same as a 70% chance the file is malicious. Isotonic calibration corrects the raw scores against held-out data so the number shown to a user carries its ordinary meaning. Measured by Brier score, where lower is better, this improved the figure from **0.0178** to **0.012**.

What the accuracy figure does and does not mean. It shows the model learned the relationship *present in this dataset* between file structure and maliciousness. It is **not** a detection rate measured against live malware. Saying "MalScan detects 98.4% of real malware" would be wrong, and the project states so in its own interface.

7 The verdict, and how it is justified

Three outcomes, not two

Verdict	Probability	Meaning
Benign	below 0.4	Nothing structurally concerning
Suspicious	0.4 – 0.65	Genuinely ambiguous — a person should look
Malicious	0.65 and above	Multiple structural indicators agree

The middle band exists deliberately. Forced into a yes/no answer, the model must commit on files it genuinely cannot separate — and legitimate packed software trips many of the same indicators as malware. "Someone should review this" is a more honest answer than a confident coin flip. Both thresholds can be moved during a scan, and the Model page shows what each setting costs in missed malware against false alarms:

Threshold	Missed malware	False alarms	Precision	Recall
0.05	9	134	0.9001	0.9926
0.1	13	60	0.9525	0.9893
0.15	15	48	0.9616	0.9877
0.2	15	28	0.9772	0.9877
0.25	18	24	0.9804	0.9852
0.3	19	20	0.9836	0.9844
0.35	22	18	0.9852	0.9819
0.4	23	14	0.9884	0.9811
0.45	25	10	0.9917	0.9795

Why this verdict: the explanation method

A bare percentage is not much use on its own, so every scan comes with an attribution built by **ablation**:

1. The file is scored normally.
2. One feature is reset to the typical value for a harmless file, with the other 72 left exactly as they are.
3. The model is asked again. However far the score moves is that feature's contribution.
4. This repeats for every feature, and the largest movers are shown as a bar chart with a plain-language summary beneath.

It is the same idea as SHAP, a standard explainability technique, but it needs no extra library and costs a single batched pass, which keeps a scan inside its one-second budget.

Two different questions. The feature importances on the Model page say what matters *generally*, across all files. The ablation says what mattered *for this one file*. They frequently disagree, and that is expected.

Which properties matter most overall

Rank	Feature	Importance
1	pe_has_version_info	0.10385
2	rule_hits	0.098
3	imp_suspicious_ratio	0.09446
4	rule_max_severity	0.08084
5	str_embedded_mz	0.06541
6	imp_suspicious_count	0.0429
7	str_suspicious_kw	0.04075
8	str_urls	0.03735
9	sec_wx_count	0.03549
10	sec_entropy_mean	0.03308

Knowing when not to answer

Every training row came from a real Windows binary whose structure parsed. A text file or a Word document has none of that, so forty-odd structural features sit at zero — a region of the problem the model has never seen. Asked anyway, it does not answer cautiously; it answers unpredictably. A harmless 166-byte text file once came back at **59% malicious**, simply because "no imports, no sections" resembles a stripped binary more than a normal one.

So MalScan stops asking. Without usable structure the file is scored on what it does have — rule matches, text indicators, overall randomness — and the verdict states which engine produced it and that the evidence is weaker.

8 Safety, privacy and speed

The safety model

- Uploaded files stay in memory as a plain byte buffer and are parsed as data. Nothing is written to an executable location, passed to a shell, imported as a library or run.
- The buffer is discarded as soon as the scan returns.
- The history database stores the analysis and the hashes — **never the file**. That is what makes it safe to keep.
- Nothing is transmitted anywhere. The VirusTotal link on a result is constructed locally from the hash; opening it is the user's choice.
- Uploads are capped at 64 MB so a single large file cannot exhaust memory.

How fast it is

File	Size	Scan time
Text, script or document	under 2 KB	1–2 ms
High-randomness blob	256 KB	~52 ms
Malformed Windows program	2 KB	~61 ms
Real system binary (full model path)	352 KB	~890 ms

Only files with a parsable Windows structure reach the trained model, which is why they cost the most. Three optimisations keep that path under a second: inference is pinned to a single thread — the forest is built for parallel training, but for a single file the coordination cost exceeded the work itself; only the parts of the file structure actually read are parsed; and the internal checksum is recomputed only when the file carries one to compare against. Together these took a real-binary scan from 1883 ms to 889 ms with byte-identical results.

Loading the model costs a further 2–5 seconds, once when the program starts. The web app pays that once per session.

9 What it is useful for

Use	How it helps
Triaging an unknown file	Someone receives an attachment or downloads an installer and wants a reasoned opinion before opening it. MalScan gives a verdict plus the evidence behind it in about a second.
Catching what signatures miss	Signature scanners only recognise malware someone has already catalogued. Because MalScan judges structure rather than exact bytes, it can flag a sample it has never seen, which is the entire argument for the approach.
Bulk checking a folder	The command-line scanner walks a directory tree and emits JSON, so it can be scripted into a larger workflow or scheduled.
Teaching and learning static analysis	Every stage is visible: the headers, the section permissions, the entropy graph, the matched rules, the per-feature contributions. It shows how the conclusion was reached rather than only stating it.
Demonstrating explainable AI	Security tools are often criticised for being unaccountable. Every verdict here can be traced to specific measurable properties of the file.
Keeping an audit trail	The scan history records what was examined and what was concluded, without ever retaining the files themselves.

Being straight about the limits

- **Static analysis cannot observe behaviour.** A file that only decrypts its payload once running leaves very little on disk to find. This is the fundamental ceiling of the approach, and the reason sandbox analysis is the most valuable thing that could be added.
- **A verdict is evidence, not proof.** Legitimate packed software trips many of the same indicators. Hence the Suspicious band.
- **Non-Windows files get weaker analysis.** The structural features are specific to Windows programs. Documents and scripts are still typed, mined and rule-matched, and the scanner says outright when a verdict rests only on those.
- **The malicious training class is synthesised.** The accuracy figure reflects a relationship learned from that data, not a field detection rate.
- **Rule matches are inputs, not conclusions.** Ordinary Windows libraries match several bundled rules, which is expected.

10 How far it could be taken

The current system is a working prototype with clear, honest boundaries. Each of the following is a genuine next step rather than a wish.

Extension	Effort	What it would add
Cross-reference with VirusTotal	Small	The SHA-256 is already computed and linked. Calling the VirusTotal API would add 70+ other engines' opinions beside MalScan's own.
Train on a real malware corpus	Medium	The training script already accepts a CSV or Parquet file with the 73 feature columns and a label. Supplying real samples would replace the synthesised malicious class and turn the accuracy figure into a genuine detection rate.
Support more file formats	Medium	The structural features are specific to Windows programs. Equivalent extractors for Android APKs, Linux ELF binaries and Office macros would extend real coverage beyond PE files.
Dynamic analysis in a sandbox	Large	Running the file in an isolated virtual machine and watching its actual behaviour would catch the one thing static analysis fundamentally cannot: a payload that only decrypts itself once running.
Deploy as a hosted service	Medium	Moving from a local app to a shared web service with accounts, quotas and an upload queue would let a whole team use one instance.
Continuous retraining	Medium	New malware appears constantly. A scheduled retraining pipeline that ingests newly labelled samples would stop the model ageing.

The single most valuable addition

Training on a real labelled malware corpus. The path is already built — the training script accepts any CSV or Parquet file carrying the 73 feature columns plus a label:

```
python -m malscan.model.train --dataset real_corpus.csv
```

Missing columns are filled with zero and reported, so a partially overlapping schema still loads. That one change would convert the accuracy figure from "the model learned this dataset" into a genuine measured detection rate — which is the main thing presently standing between this prototype and a deployable tool.

11 Questions you should expect

Short, direct answers to the questions most likely to be asked about this project.

Why machine learning instead of a signature scanner?

A signature only matches malware someone has already seen and catalogued. It cannot recognise a new variant. A model trained on structural properties does not need the specific sample, only the shape of it, so it can flag something genuinely new.

Is it safe to scan real malware with this?

Yes. The file is read into memory as plain data and parsed. It is never written to an executable location, never passed to a shell, never loaded as a library and never run. Analysing a sample is about as risky as looking at a photograph of one.

What is entropy, in one sentence?

A measure of how unpredictable the bytes are, from 0 to 8. Normal code and text score low because they repeat; compressed or encrypted data scores near 8 because it looks random, which is why a high value suggests something is being hidden.

Why three verdicts instead of malicious or clean?

Some files genuinely sit on the boundary, and legitimate packed software trips many of the same indicators as malware. Forcing a binary answer on those produces a confident guess. Saying 'a person should look at this' is more honest and more useful.

Why was Random Forest chosen over XGBoost?

Models are selected on ROC-AUC, which does not depend on a threshold. Random Forest scored 0.9963 against XGBoost's 0.9962 — very close, and the selection rule picked the higher one. XGBoost was actually better on raw accuracy, and the comparison is recorded rather than hidden.

What does calibration mean here?

A raw forest vote of 0.8 does not reliably mean an 80% chance of being malicious. Isotonic calibration corrects those scores against held-out data so the number shown to a user carries its plain meaning. It improved the Brier score from 0.0178 to 0.012, where lower is better.

Why is the accuracy figure not a detection rate?

The malicious half of the training data is synthesised from documented behaviour profiles, not collected from live samples. The figure shows the model learned the relationship present in that data. It is not a measured detection rate against real malware in the wild, and should never be quoted as one.

What stops the model being tested on files it trained on?

Several rows descend from the same source binary. The split is done by source binary rather than by row, so no binary appears on both sides. The trainer asserts this and records 0 leaked files.

What happens to a file that is not a Windows program?

The model is not asked, because it was trained only on parsable Windows executables and would be guessing outside its experience. Such files are scored from content signals instead — rules, strings, entropy — and the verdict states which engine produced it and that the evidence is weaker.

What is the single biggest limitation?

Static analysis cannot observe behaviour. A file that only decrypts its payload once running leaves very little on disk to detect. That is why sandbox analysis is the most valuable future addition.

12 Quick reference

Terms in one line each

Term	Meaning
PE (Portable Executable)	The file format of every Windows <code>.exe</code> and <code>.dll</code> .
Entropy	How unpredictable the bytes are, 0–8. High suggests compression or encryption — something hidden.
Packing	Compressing or encrypting a program so its real contents are not visible until it runs. Used legitimately too, which is why it is a hint, not proof.
Imports	The Windows functions a program borrows. Which ones it asks for says a lot about what it intends to do.
Section	A named region of a program — code, data, resources — each with permissions to read, write or execute.
Overlay	Data appended past the program's proper end. A common hiding place.
YARA	The industry-standard language for writing malware pattern rules.
ROC-AUC	How well a model ranks malicious above benign, independent of any threshold. 1.0 is perfect, 0.5 is guessing.
Calibration	Adjusting raw scores so a stated probability means what it says.
Ablation	Changing one input and re-measuring, to find out how much that input mattered.
False positive	A harmless file wrongly flagged as malicious.
False negative	A malicious file wrongly cleared.

Where everything lives

Path	Responsibility
<code>app.py</code>	Web interface entry point
<code>malscan_cli.py</code>	Command-line scanner
<code>malscan/config.py</code>	Every path, limit and threshold — no magic numbers elsewhere
<code>malscan/scanner.py</code>	The scan pipeline; the single function the interface calls
<code>malscan/features/</code>	The 73-feature contract and all the extractors
<code>malscan/rules/</code>	The 27 YARA rules and the matching engine
<code>malscan/model/</code>	Dataset building, training, inference, explanation
<code>malscan/storage/</code>	SQLite scan history

Path	Responsibility
<code>malscan/ui/</code>	The six interface pages
<code>tests/test_malscan.py</code>	31 tests, runnable without pytest

Running it

`pip install -r requirements.txt` — install dependencies

`python -m malscan.model.train` — build the model

`streamlit run app.py` — launch the web interface

`python malscan_cli.py file.exe` — scan from the command line

`python tests/test_malscan.py` — run the test suite

The app runs before training too; it falls back to a transparent heuristic scorer and says so on every verdict rather than pretending there is a model behind it.